

OMNICART MARKETPLACE PROGRAM

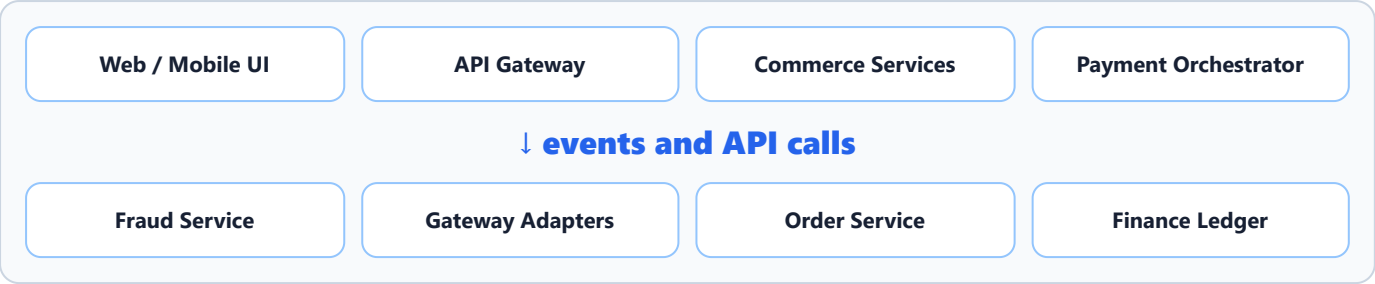
High Level Design Document

Architecture, integration, deployment, security, observability, and scalability design for marketplace commerce and payment orchestration.

Project:	OmniCart Commerce Platform with Multi-Gateway Payments
Prepared For:	Product, Engineering, QA, Security, Operations, Finance
Generated:	2026-05-13
Classification:	Synthetic test data for ingestion pipeline validation
Version:	1.0

1. Architecture Overview

OmniCart uses a modular service architecture with a React web frontend, API gateway, commerce domain services, payment orchestration service, event bus, relational data stores, search cluster, cache, fraud engine, gateway adapters, and observability stack.



2. Major Components

Component	Responsibility	Technology	Scaling Model
Storefront Web	Customer shopping, cart, checkout, account	React, CDN, edge cache	Static CDN plus API calls
API Gateway	Auth, rate limiting, routing, request policy	Kong / AWS API Gateway	Horizontal
Catalog Service	Product metadata, categories, seller offers	Java/Kotlin, Postgres, OpenSearch	Read replicas and search shards
Cart Service	Cart state, price preview, promotions	Node.js, Redis, Postgres	Redis cluster
Checkout Service	Order intent, address, tax, delivery promise	Java, Postgres	Horizontal with optimistic locks
Payment Orchestrator	Gateway routing, payment lifecycle, webhooks	Java, Postgres, Kafka	Partition by paymentIntentId
Gateway Adapter	Gateway-specific payloads and signatures	Java workers	Per gateway worker pool
Finance Ledger	Double-entry accounting and settlement	Postgres, batch exports	Append-only partitioned tables

3. Payment Routing Design

The gateway routing engine evaluates gateway availability, payment method, merchant account, BIN, issuer, risk score, retry attempt, gateway cost, and previous decline reason. Routing decisions are deterministic and logged with rule version.

```
Input: tender=CARD, bin=411111, amount=12999, risk=31, gatewayHealth={RZP:green, STRIPE:amber}
Rules:
  1. If tender is UPI -> prefer PAYU_UPI unless health red
  2. If card domestic and amount < 50000 -> prefer RAZORPAY
  3. If retry after timeout -> use alternate gateway
Output: routeId=pay-route-v17-card-domestic, sequence=[RAZORPAY, STRIPE]
```

4. Integration Diagram

```

Checkout Service
  | createPaymentIntent()
  v
Payment Orchestrator ---- riskCheck() ---- Fraud Service
  | authorize()
  v
Gateway Adapter ---- HTTPS ---- External Gateway
  ^                               |
  | webhook                      v
Webhook Receiver ---- Kafka payment.events ---- Order / Ledger / Notification

```

5. External APIs and Endpoints

Endpoint	Consumer	Purpose	SLO
POST /api/v1/cart/items	Storefront	Add SKU to cart	p95 300ms
POST /api/v1/checkout/order-intents	Storefront	Create order intent	p95 700ms
POST /api/v1/payments/intents	Checkout	Create payment intent	p95 450ms
POST /api/v1/payments/{id}/authorize	Storefront	Authorize payment	p95 900ms excl. gateway
POST /api/v1/payments/webhooks/{gateway}	Gateway	Payment event ingestion	p95 250ms
POST /api/v1/refunds	OMS / Support	Create refund request	p95 600ms

6. Security Architecture

- Raw PAN is never stored; card data is tokenized by PCI-compliant gateway or token vault.
- Gateway webhooks require HMAC signature validation and IP allowlist where supported.
- Customer JWT authorizes customer-facing operations; service-to-service traffic uses mTLS.
- Payment and refund APIs enforce idempotency and rate limits.
- Audit events include actor, customer id, payment intent id, previous state, new state, and correlation id.

7. Observability

Metric	Dimension	Alert
payment_authorization_success_rate	gateway, tender, issuer	< 94% for 10 min
gateway_timeout_rate	gateway	> 2% for 5 min
webhook_processing_lag_seconds	gateway	> 60s p95
ledger_mismatch_count	settlement file	> 0 critical

8. Resiliency Patterns

- Gateway adapter circuit breakers open after configured timeout/error thresholds.
- Status inquiry jobs reconcile UNKNOWN payment attempts.
- Webhook processing is idempotent and retryable through Kafka dead-letter queues.
- Order confirmation waits for payment state machine event rather than browser redirect alone.
- Finance ledger writes are append-only and occur in the same transaction as final payment state where feasible.